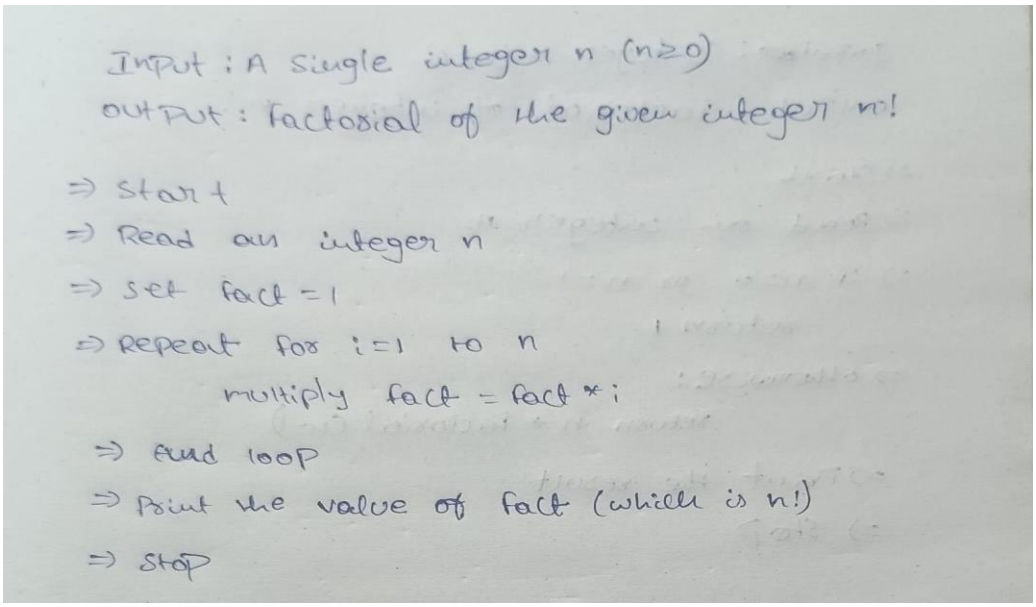
PRACTICAL-04

AIM: Implementation and Time analysis of factorial program using iterative and recursive method.

FACTORIAL BY ITERATIVE

ALGORITHM:



PROGRAM:

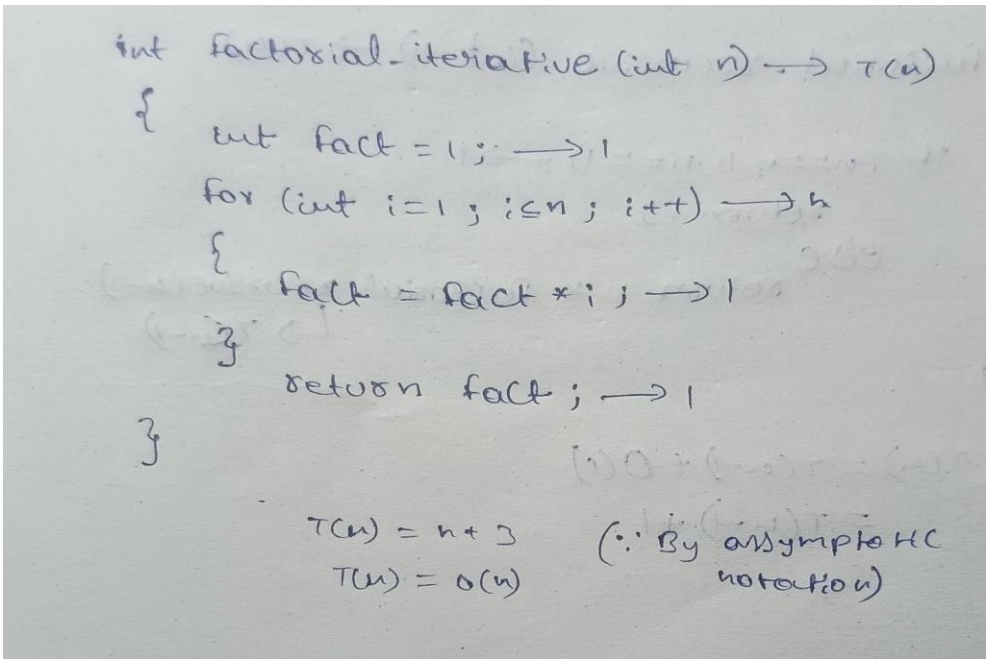
```
#include <iostream>
using namespace std;
int factorial_iterative(int n) {
    int fact = 1;
    for (int i = 1; i <= n; i++) {
        fact =fact*i;
    }
    return fact;
}
```

```
int main() {
    int n;
    cout << "Enter a number: ";
    cin >> n;
    cout << "Factorial (Iterative): " << factorial_iterative(n) << endl;
    return 0;
}
```

OUTPUT:

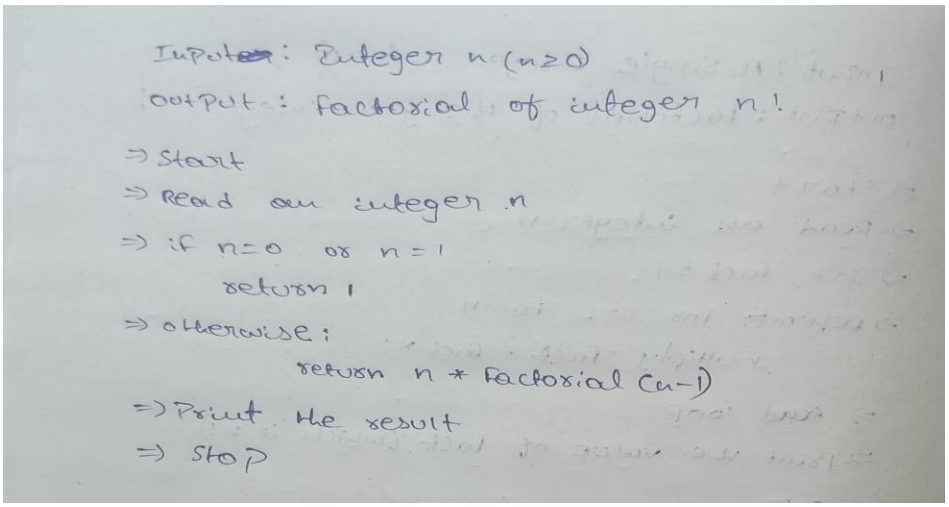
```
Enter a number: 5
Factorial (Iterative): 120
```

TIME COMPLEXITY:



FACTORIAL BY RECURSIVE

ALGORITHM:



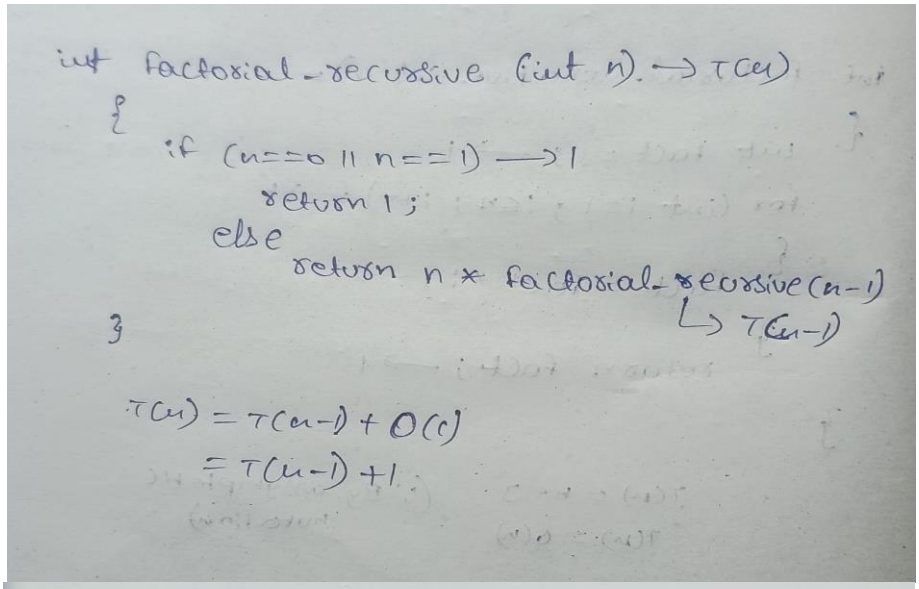
PROGRAM:

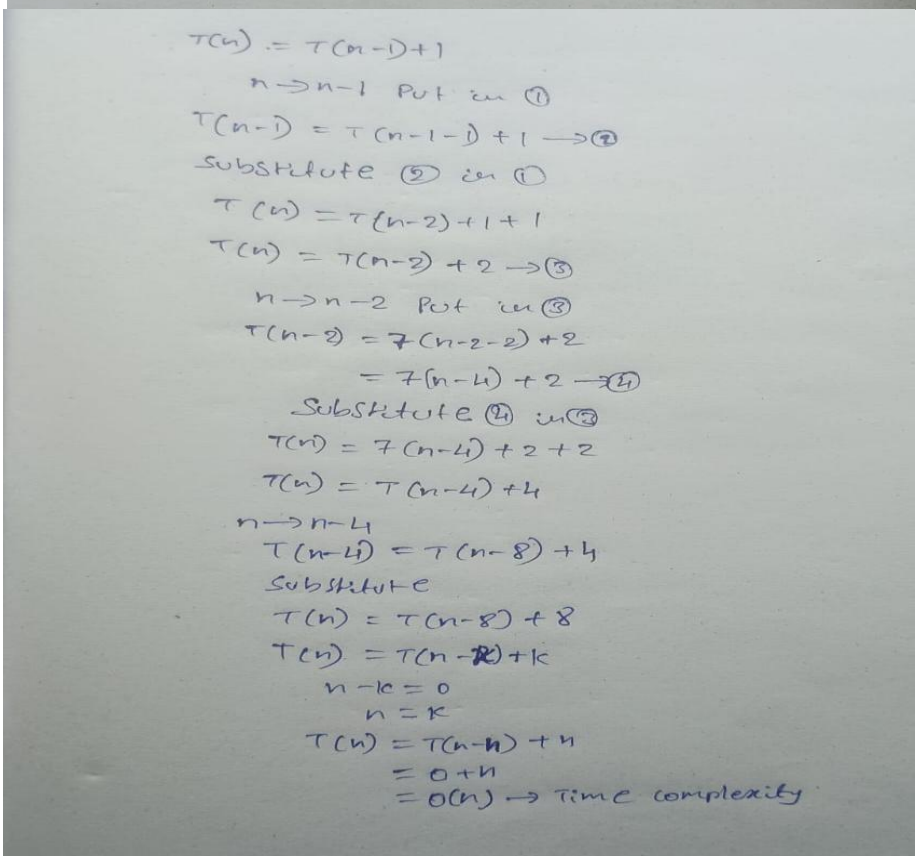
```
#include <iostream>
using namespace std;
int factorial_recursive(int n) {
    if (n == 0 || n == 1)
        return 1;
    else
        return n * factorial_recursive(n - 1);
}
int main() {
    int n;
    cout << "Enter a number: ";
    cin >> n;
    cout << "Factorial (Recursive): " << factorial_recursive(n) << endl;
    return 0;
}
```

OUTPUT:

```
Enter a number: 5
Factorial (Recursive): 120
```

TIME COMPLEXITY:





CONCLUSION:

The factorial program demonstrates both iterative and recursive approaches. Iterative factorial runs in linear time $O(n)$with constant space, making it efficient. Recursive factorial also has $O(n)$time but requires $O(n)$stack space due to repeated function calls. Thus, iteration is more memory-efficient, while recursion is conceptually elegant.

Y.Sudheer Kumar (92400118922)